# Part A:

## Problem Definition:

1. Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.

### Solution:

```java
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.List;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}
class SignupForm {

    @NotBlank
    private String username;

    @NotBlank
    @MaxLength(20)
    private String password;

    @NotBlank
    @MaxLength(50)
    private String email;

    SignupForm(String username, String password, String email) {
        this.username = username;
        this.password = password;
        this.email = email;
    }
}

class FormValidator {

    public static List<String> validate(Object obj) {
        List<String> errors = new ArrayList<>();

        Class<?> clazz = obj.getClass();
        Field[] fields = clazz.getDeclaredFields();

        for (Field field : fields) {
            field.setAccessible(flag: true);

            try {
                Object value = field.get(obj);

                if (field.isAnnotationPresent(annotationClass: NotBlank.class)) {
                    if (value == null || value.toString().trim().isEmpty()) {
                        errors.add(field.getName() + " must not be blank");
                    }
                }

                if (field.isAnnotationPresent(annotationClass: MaxLength.class)) {
                    MaxLength annotation =
                            field.getAnnotation(annotationClass: MaxLength.class);

                    int max = annotation.value();
                    if (value != null &&
                        value.toString().length() > max) {
                        errors.add(
                            field.getName() +
                            " must not exceed " +
                            max +
                            " characters"
                        );
                    }
                }
            } catch (IllegalAccessException e) {
                errors.add("Cannot access field: " + field.getName());
            }
        }
        return errors;
    }
}
public class Main {
    Run | Debug
    public static void main(String[] args) {
        SignupForm form = new SignupForm(
            username: "",
            password: "thispasswordiswaytoolong123456",
            email: "student@example.com"
        );
        List<String> errors = FormValidator.validate(form);
        if (errors.isEmpty()) {
            System.out.println(x: "Form is valid!");
        } else {
            for (String error : errors) {
                System.out.println(error);
            }
        }
    }
}
```

## Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPS-SEM3-2> javac Main.java
PS C:\Users\parma\OneDrive\Documents\OOPS-SEM3-2> java Main
username must not be blank
password must not exceed 20 characters
PS C:\Users\parma\OneDrive\Documents\OOPS-SEM3-2>
```

## Problem Definition:

2. Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.

## Solution:

```java
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}
class MyTests {
    @Run
    public void testAddition() {
        System.out.println(x: "Addition test passed");
    }
    @Run
    public void testSubtraction() {
        System.out.println(x: "Subtraction test passed");
    }
    public void normalMethod() {
        System.out.println(x: "This method should not run");
    }
    @Run
    public void testMultiplication() {
        System.out.println(x: "Multiplication test passed");
    }
}

public class MiniTestRunner {
    Run | Debug
    public static void main(String[] args) throws Exception {

        MyTests testObject = new MyTests();

        Method[] methods = MyTests.class.getDeclaredMethods();

        int count = 0;

        for (Method method : methods) {

            if (method.isAnnotationPresent(annotationClass: Run.class)
                    && method.getParameterCount() == 0) {

                method.invoke(testObject);
                count++;
            }
        }
        System.out.println("Total methods run: " + count);
    }
}
```

## Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPS-SEM3-2> javac MiniTestRunner.java
PS C:\Users\parma\OneDrive\Documents\OOPS-SEM3-2> java MiniTestRunner
Subtraction test passed
Multiplication test passed
Addition test passed
Total methods run: 3
```

## Problem Definition:

3. Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.

## Solution:

```java
import java.lang.annotation.*;
import java.lang.reflect.Field;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}
class Student {
    @Column(name = "student_id")
    private int id;

    @Column(name = "student_name")
    private String name;

    @Column(name = "email")
    private String email;

    public void display() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Email: " + email);
    }
}
public class ColumnMapper {
    Run | Debug
    public static void main(String[] args) throws Exception {

        String[] headers = {
            "student_id",
            "student_name",
            "email"
        };

        String[] data = {
            "101",
            "Vaidehi",
            "vaidehi@gmail.com"
        };
        Student student = new Student();
        for (Field field : Student.class.getDeclaredFields()) {
            if (field.isAnnotationPresent(annotationClass: Column.class)) {

                Column column = field.getAnnotation(annotationClass: Column.class);
                String columnName = column.name();

                int index = -1;

                for (int i = 0; i < headers.length; i++) {
                    if (headers[i].equals(columnName)) {
                        index = i;
                        break;
                    }
                }

                if (index == -1) {
                    System.out.println(
                        "Missing column: " + columnName
                    );
                    continue;
                }
                field.setAccessible(flag: true);
                if (field.getType() == int.class) {
                    field.setInt(student, Integer.parseInt(data[index]));
                } else if (field.getType() == String.class) {
                    field.set(student, data[index]);
                }
            }
        }
        student.display();
    }
}
```

## Output:

```
PS C:\Users\parma\OneDrive\Documents\OOPS-SEM3-2> javac ColumnMapper.java
PS C:\Users\parma\OneDrive\Documents\OOPS-SEM3-2> java ColumnMapper
ID: 101
Name: Vaidehi
Email: vaidehi@gmail.com
```